What Does LoRA Change?

An interpretability study of Low-Rank Adaptation in a small LLM

Marcos Lahoz

May 29, 2026

Abstract

Low-Rank Adaptation (LoRA) fine-tunes large language models by learning small low-rank updates to frozen weight matrices. It is ubiquitous, yet it is rarely asked *what* a LoRA adapter actually changes inside the model. We study this on `Qwen2.5-1.5B` instruction-tuned on Alpaca, with two complementary, training-free diagnostics: (i) the *relative update magnitude* $\|\Delta W\|/\|W\|$ and the effective rank of ΔW for every adapted projection, revealing where LoRA concentrates its capacity; and (ii) the *representation drift* between the adapter-enabled and adapter-disabled model, layer by layer, revealing how far the internal representations move and at what depth. We run these across an ablation over the LoRA rank and the set of adapted modules. The framework turns "LoRA works" into measurable, localised statements about a fine-tuned model.

1 Introduction

LoRA [1] freezes a pretrained weight $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and learns a low-rank update $\Delta W = \frac{\alpha}{r} BA$ with $B \in \mathbb{R}^{d_{\text{out}} \times r}$, $A \in \mathbb{R}^{r \times d_{\text{in}}}$ and $r \ll d$. It is the default way to fine-tune LLMs cheaply, but it is usually treated as a black box. We ask three questions:

1. **Where** does LoRA put its capacity — which layers and which projection types (q, k, v, o vs. the MLP gate, up, down) receive the largest updates?
2. **How much** do the model's hidden representations actually move once the adapter is enabled, and at which depth?
3. How do (1) and (2) depend on the LoRA **rank** and the set of **adapted modules**?

The analyses are training-free post-hoc diagnostics and apply to any LoRA-adapted transformer.

2 Method

(1) Effective update magnitude. For each adapted module we form the explicit update $\Delta W = \frac{\alpha}{r} BA$ and report the relative Frobenius norm

$$\rho \;=\; \frac{\|\Delta W\|_F}{\|W\|_F},$$

i.e. how large the change is compared with the original weight. We also summarise the *shape* of ΔW by the participation ratio of its singular values $\{\sigma_i\}$,

$$\text{eff-rank}(\Delta W) \;=\; \exp\Big(-\sum_i p_i \log p_i\Big), \qquad p_i = \frac{\sigma_i}{\sum_j \sigma_j},$$

which measures how many directions the update effectively uses (bounded by r).

(2) Representation drift. Let $h_{\text{on}}^{(\ell)}(x)$ and $h_{\text{off}}^{(\ell)}(x)$ be the mean-pooled hidden state at layer ℓ for input x with the adapter enabled and disabled (the rest of the model is identical). Over a held-out set $\mathcal{X}$ we report, per layer, the mean cosine similarity and the relative L_2 drift

$$\cos^{(\ell)} = \frac{1}{|\mathcal{X}|}\sum_{x} \cos\left(h_{\text{on}}^{(\ell)}, h_{\text{off}}^{(\ell)}\right), \qquad \delta^{(\ell)} = \frac{1}{|\mathcal{X}|}\sum_{x} \frac{\|h_{\text{on}}^{(\ell)} - h_{\text{off}}^{(\ell)}\|}{\|h_{\text{off}}^{(\ell)}\|}.$$

This isolates the causal effect of the adapter on the representations.

3 Experimental setup

We fine-tune the base `Qwen2.5-1.5B` on a subset of Alpaca with an instruction template, training only the LoRA adapters. We ablate the rank $r \in \{4, 16, 64\}$ and the adapted module set (*attn*: q, k, v, o; *all*: $+\,\text{gate}, \text{up}, \text{down}$). Representation drift is measured on a fixed set of held-out instructions. All code, the ablation runner and the plots are in the repository.

4 Results

[Figure `update_heatmap_r16_all.png` is generated by the study scripts; run scripts/run_ study.py to produce it.]

[Figure `update_by_layer_r16_all.png` is generated by the study scripts; run scripts/run_ study.py to produce it.]

[Figure `drift_r16_all.png` is generated by the study scripts; run scripts/run_ study.py to produce it.]

Table 1 (produced as `results.csv`) summarises the ablation: the average relative update, its split between attention and MLP projections, the mean effective rank of ΔW, and the final-layer representation drift.

rank	modules	mean ρ	ρ (attn)	ρ (mlp)	drift (rel. L_2, final)
4	all	...	...	...	...
16	all	...	...	...	...
64	all	...	...	...	...

Table 1: Ablation summary (fill from `results.csv` after running).

5 Discussion and future work

The two diagnostics localise fine-tuning: ρ shows which weights move, the effective rank shows how low-dimensional the change is, and the drift curve shows where in the network behaviour is reshaped (typically the later layers). Natural extensions: (i) *behavioural* validation linking drift to task accuracy; (ii) trained linear *probes* on the representations before/after, connecting to structural-probe methodology; (iii) scaling to 7B models and to DoRA / other PEFT variants; (iv) merging adapters and tracking weight-space directions.

References

[1] Hu, E. et al. (2021). *LoRA: Low-Rank Adaptation of Large Language Models.* arXiv:2106.09685.

[2] Liu, S.-Y. et al. (2024). *DoRA: Weight-Decomposed Low-Rank Adaptation.* ICML.

[3] Hewitt, J. & Manning, C. (2019). *A Structural Probe for Finding Syntax in Word Representations.* NAACL.

[4] Qwen Team (2024). *Qwen2.5 Technical Report.*